
MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent

Hongli Yu^{1,2,3}, Tinghong Chen², Jiangtao Feng², Jiangjie Chen^{1,3}, Weinan Dai^{1,2,3},
Qiyong Yu^{1,2,3}, Ya-Qin Zhang^{2,3}, Wei-Ying Ma^{2,3}, Jingjing Liu^{2,3}, Mingxuan Wang^{1,3}, Hao Zhou^{2,3}

¹ByteDance Seed ²Institute for AI Industry Research (AIR), Tsinghua University

³SIA-Lab of Tsinghua AIR and ByteDance Seed

Why long-context reasoning is still hard

现实中的长文本任务很多：读整本书、长链条推理、管理 agent 的长期记忆，这些都会很快超出当前 LLM 的典型上下文窗口。

尽管已经有长度外推、高效注意力和 memory 模块等方向的改进，但作者认为，在不明显掉性能的前提下，以线性复杂度处理几乎无限长文本，仍然是长文本处理中的核心难题。

因此，一个真正强的长上下文系统，应该同时满足三点：

- 能处理几乎无限长的输入；
- 随着长度增长，性能不要明显崩掉；
- 推理复杂度尽量保持线性。

已有工作：

长度外推：通过调整位置编码或继续长上下文训练来扩展窗口，但在极长文本上仍可能性能下降，而且速度仍受二次复杂度限制。

稀疏/线性注意力：通过改 attention 结构降低复杂度，但通常需要训练新架构；线性注意力有并行训练困难，稀疏注意力往往依赖预定义模式。

上下文压缩 / 外部 memory 模块：通过压缩输入或外挂 memory 来减轻负担，但常常需要额外模块或额外上下文操作，影响兼容性和并行化。

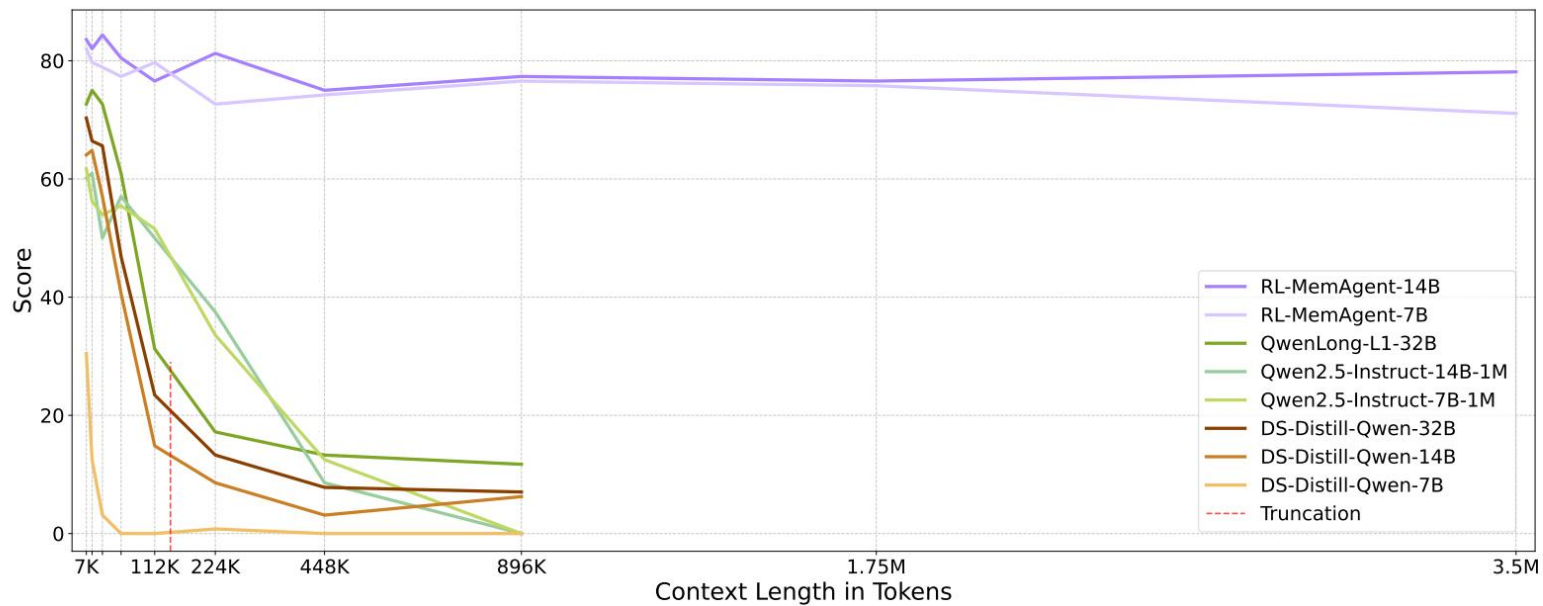


Figure 1 Accuracy scores of RULER-HotpotQA [1, 2]. Even models that employ long-context continual pretraining and extrapolation techniques fail to maintain consistent performance. In contrast, MEMAGENT with RL demonstrates nearly lossless performance extrapolation.

MemAgent 不再继续改 attention，而是把长文本处理改写为分块读取、逐步记忆。

模型每次读取一个 chunk，并更新一个固定长度的文本记忆。目标是同时实现：超长输入、性能稳定、线性成本。

MemAgent 把长文档看作一个按顺序到来的证据流。
在每一步，模型只看到两样东西：当前 chunk，以及一个固定长度的 memory。
读完当前 chunk 后，模型会用新的内容覆盖式更新 memory。
当所有 chunk 都处理完之后，再基于最终 memory 生成最终 memory 生成答案。

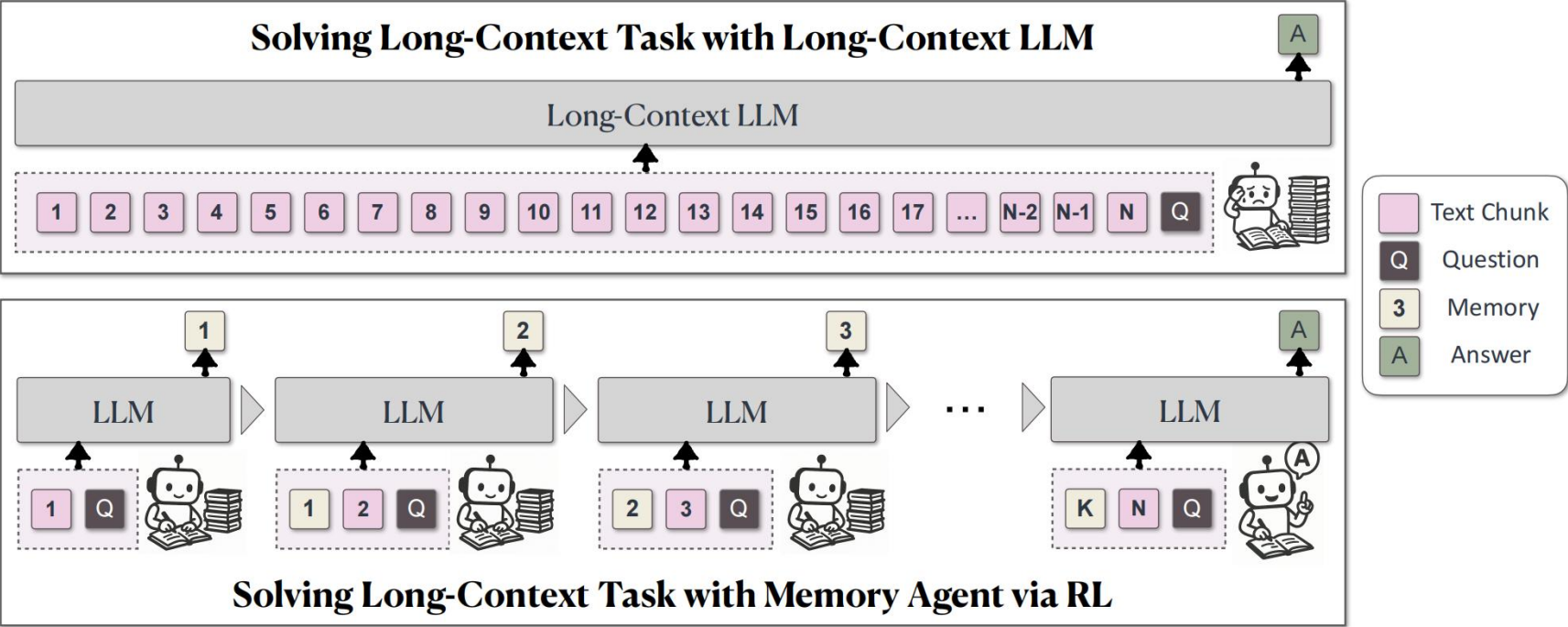


Figure 2 MEMAGENT is inspired by the way humans process long documents. It divides the document into multiple chunks and allows LLMs to process them iteratively, recording relevant information in memory. Finally, LLMs generate answers based on the information stored in the memory.

Multi-Conv DAPO:

对同一个样本，模型会产生多段 context-independent conversations:

前面多轮用于 memory update，最后一轮用于 answer generation，并且将每个 conversation 单独视为优化目标。

对于同一个样本，先由最后一轮 answer conversation 计算 reward。

$$R(\hat{y}, Y) = \max_{y \in Y} (\mathbb{I}(\text{is_equiv}(y, \hat{y}))) \quad R(\hat{y}, Y) = \frac{|\{y \in Y \mid \mathbb{I}(y \in \hat{y})\}|}{|Y|}$$

再将该 reward / advantage 分发给该样本对应的全部 conversations

$$\hat{A}_{i,j,t} = r_i - \text{mean}(\{R_i\}_{i=1}^G)$$

loss 定义整体仍然沿用 DAPO 的 clipped objective + KL penalty。

但 loss 的求和维度从传统的 (group, token)，扩展成 (group, conversation, token)。

作者 follow DrGRPO 的设置，没有再用 reward 的标准差去归一化 advantage。

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_{i,j}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q, o_{i,j-1})} \left[\frac{1}{\sum_{i=1}^G \sum_{j=1}^{n_i} |o_{i,j}|} \sum_{i=1}^G \sum_{j=1}^{n_i} \sum_{t=1}^{|o_{i,j}|} \left(\mathcal{C}_{i,j,t} - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right) \right]$$

$$\text{where } \mathcal{C}_{i,j,t} = \min \left(r_{i,j,t}(\theta) \hat{A}_{i,j,t}, \text{clip} \left(r_{i,j,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}} \right) \hat{A}_{i,j,t} \right)$$

实验结果：

训练与主评测任务都基于 长文本 QA
训练数据由 HotpotQA 构造得到：每个样本约 200 篇文章、28K tokens，一共处理了 80,000 个样本，过滤后保留 32,768 个训练样本。
验证集由 HotpotQA val split 构造；测试长度从 7K 一直到 3.5M tokens。

backbone: Qwen2.5-7B-Instruct / 14B-Instruct
训练时限制为 8K context window
8K: 1024 query + 5000 chunk + 1024 memory + 1024 output
一个样本通常需要 5-7 轮 conversations

Table 2 Main experimental results comparing model performance across various context lengths. All values represent accuracy (%).

Model	Length									
	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
QwenLong-L1-32B	72.66	75.00	72.66	60.94	31.25	17.19	13.28	11.72	N/A	N/A
Qwen2.5-Instruct-14B-1M	60.16	60.94	50.00	57.03	50.00	37.50	8.59	0.00	N/A	N/A
Qwen2.5-Instruct-7B-1M	61.72	56.25	53.91	55.47	51.56	33.59	12.50	0.00	N/A	N/A
DS-Distill-Qwen-32B	70.31	66.41	65.62	46.88	23.44	13.28	7.81	7.03	N/A	N/A
DS-Distill-Qwen-14B	64.06	64.84	57.03	40.62	14.84	8.59	3.12	6.25	N/A	N/A
DS-Distill-Qwen-7B	30.47	12.50	3.12	0.00	0.00	0.78	0.00	0.00	N/A	N/A
RL-MEMAGENT-14B	83.59	82.03	84.38	80.47	76.56	81.25	75.00	77.34	76.56	78.12
RL-MEMAGENT-7B	82.03	79.69	78.91	77.34	79.69	72.66	74.22	76.56	75.78	71.09

baseline性能退化很明显
RL-MemAgent 在 7K-896K 区间表现明显更稳定，进一步外推到 1.75M / 3.5M 仍保持较高准确率

RL 训练是否必要？

作者在这里做了一个三组比较：原始 Qwen2.5-Instruct、带 memory 但不做 RL 的版本，以及 RL-MemAgent。

原始 backbone 随长度增长迅速退化，尤其在 112K 之后更明显。

仅加入 memory、但不做 RL，性能有所改善，但仍会持续下降。

加入 RL 后，性能在各个长度上都更稳定。

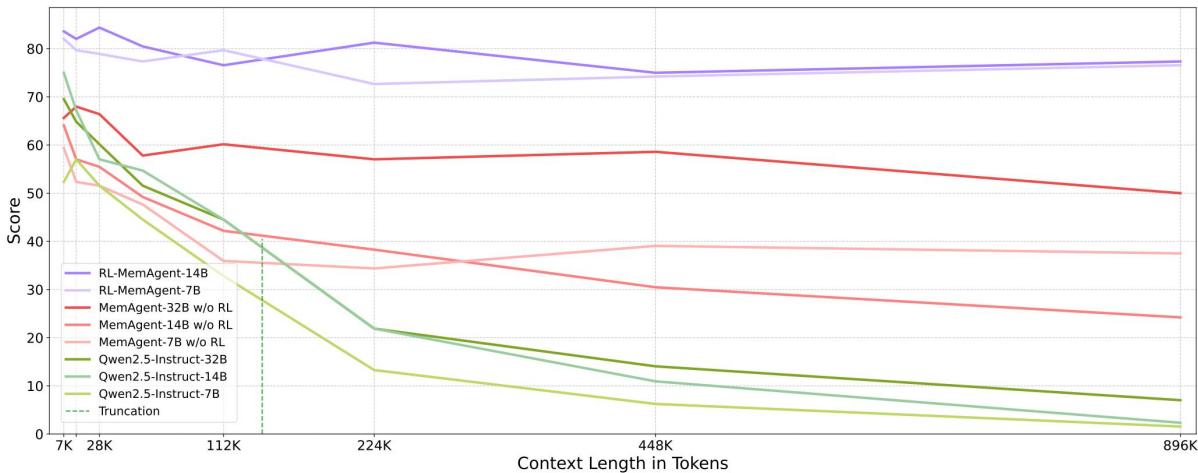
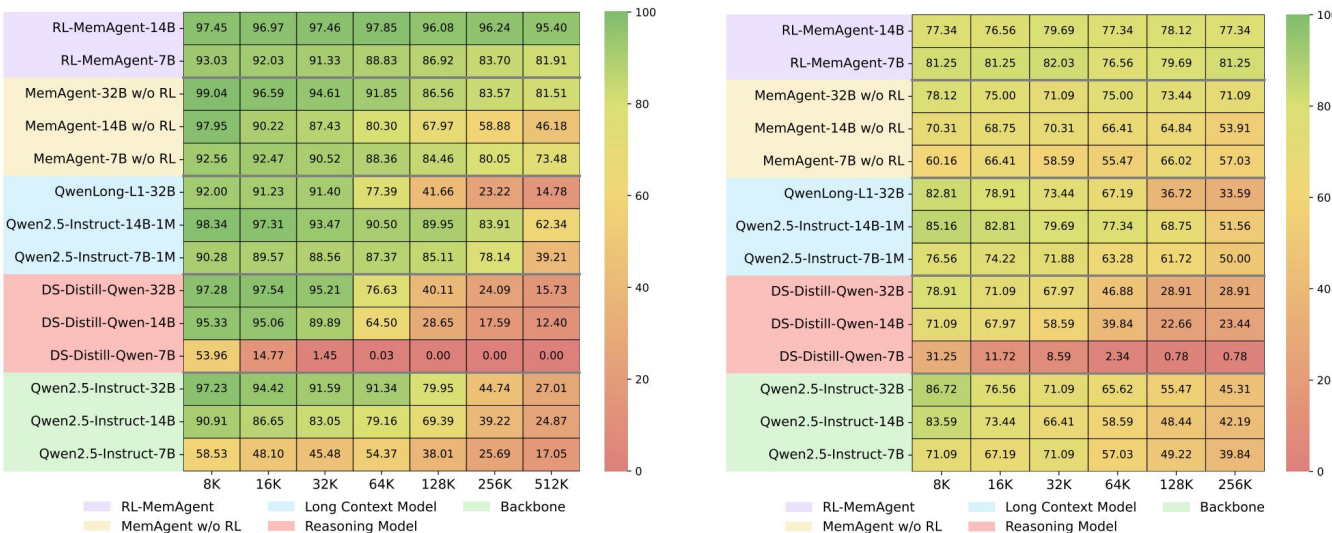


Figure 5 Ablation study on RULER-HotpotQA comparing models with and without RL training across context lengths from 28K to 896K tokens.

OOD 任务上的泛化能力：

作者在这里用的是 RULER benchmark 上的一系列 OOD 任务，上下文长度通常覆盖 8K 到 512K，而 SQuAD 由于原始文档长度限制，只做到 256K。

Figure 6 的整体趋势非常一致：MemAgent 在不同任务类别上都保持了较好的稳定性。



(a) RULER average across 10 tasks (b) RULER-QA task from SQuAD

Figure 6 Performance heatmaps on RULER benchmark tasks showing accuracy scores across different context lengths (greener indicates better performance). Models are grouped by type on the vertical axis. (a) Average performance across 10 synthetic tasks including needle-in-a-haystack variants, variable tracking, and word extraction. (b) Question answering task synthesized from SQuAD dataset with context lengths up to 256K tokens.

When to Memorize and When to Stop: Gated Recurrent Memory for Long-Context Reasoning

**Leheng Sheng^{1,2,‡}, Yongtao Zhang¹, Wenchang Ma¹, Yaorui Shi³, Ting Huang¹,
Xiang Wang³, An Zhang^{3,†}, Ke Shen^{1,†}, Tat-Seng Chua²**

¹Bytedance Seed, ²National University of Singapore,

³University of Science and Technology of China

[‡]Work done at ByteDance Seed, [†]Corresponding authors

GRU-Mem 建立在 MemAgent 的 recurrent memory 范式之上

作者指出，MemAgent 仍有两个关键问题：

1. Memory Explosion: 即使当前 chunk 没有证据，也可能继续更新 memory，导致噪声不断累积
2. Lack of Exit Mechanism: 即使已经收集到足够证据，仍然必须把剩余 chunks 全部处理完

这两个问题同时影响：

- 记忆稳定性
- 推理效率

因此，GRU-Mem 的目标是：

让模型学会“什么时候该更新，什么时候该停止”

workflow:

借鉴 GRU 的 gating 思想, GRU-Mem 在 recurrent workflow 中加入两个 gate:

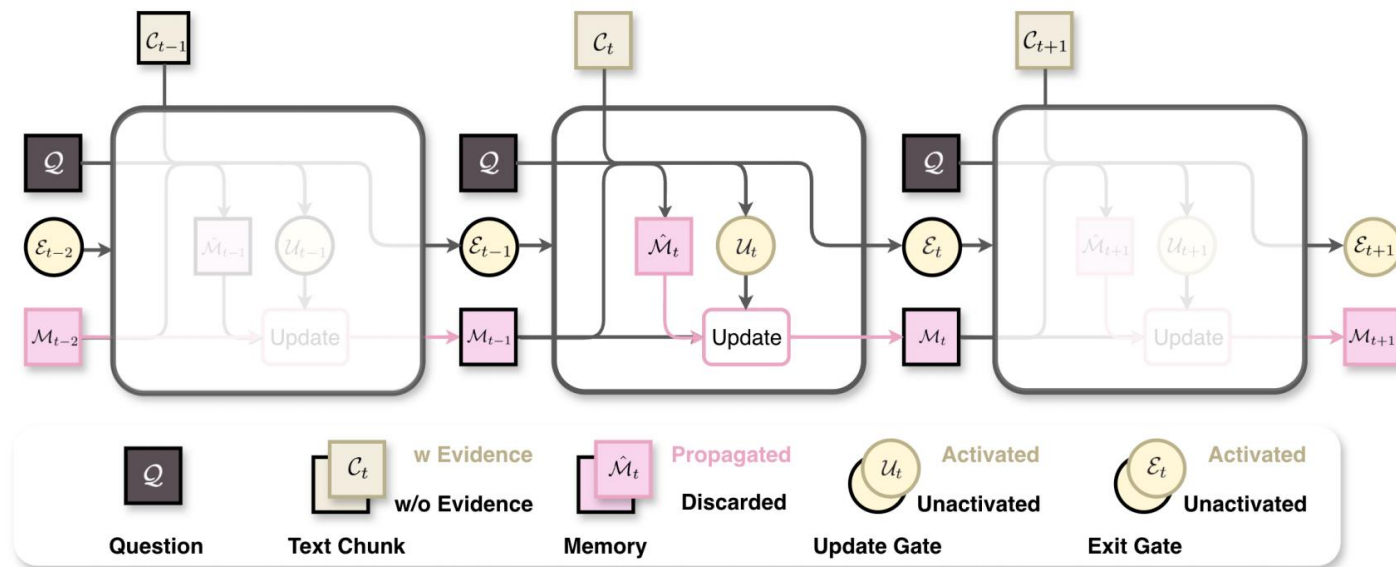
- Update Gate (UG): 决定当前 chunk 是否值得更新 memory ;

- Exit Gate (EG): 决定是否可以直接提前终止整个流程
若 $UG = \text{True}$, 则更新 memory ; 否则保留旧 memory ;

若 $EG = \text{True}$, 则说明证据已足够, 可以直接停止并进入 answer generation ;

这样同时提升:

- memory stability
- inference efficiency



You should reason about whether the new section contains useful information, what to update, and what to do next first between `<think>` and `</think>`.

If the new section contains useful information about the problem, you should first generate `<check>yes</check>`. After that, update the new memory between `<update>` and `</update>`.

If the new section does not contain useful information about the problem, you should first generate `<check>no</check>`. After that, you should keep the previous memory unchanged between `<update>` and `</update>`.

In the end, if you haven't collected enough information for the problem, return `<next>continue</next>`. ONLY when enough information is collected, return `<next>end</next>`.

Reward:

Outcome reward.监督模型最终是否基于记忆给出正确答案

$$r^{\text{outcome}} = \mathbb{I}(\text{is_equiv}(\mathcal{A}, \hat{\mathcal{A}})),$$

Update reward.监督 update gate 是否在当前 chunk 含有关键证据时更新、无关时跳过。

$$r_t^{\text{update}} = \begin{cases} 1, & \mathcal{U}_t \text{ is correct} \\ -1, & \mathcal{U}_t \text{ is incorrect.} \end{cases}$$

Exit reward.监督 exit gate 是否在“最后一个必要证据”处正确退出

$$r^{\text{exit}} = \begin{cases} -0.75, & t_{\text{exit}} < t_{\text{last evidence}} \\ 0, & t_{\text{exit}} = t_{\text{last evidence}} \\ -0.5, & t_{\text{exit}} > t_{\text{last evidence}}. \end{cases}$$

Format reward. To ensure that the generation $o_{g,t}$ of the memory agent ϕ_θ can be parsed correctly, we introduce an additional format reward r_{format} . We check whether the format meets the requirement of enclosed sequence of `<think> </think>`, `<check> </check>`, `<update> </update>`, and `<next> </next>`. Additionally,

最终整条轨迹的reward:

$$r_g^{\text{traj}} = r_g^{\text{outcome}} + r_g^{\text{exit}} + r_g^{\text{format}}.$$

advantage设计:

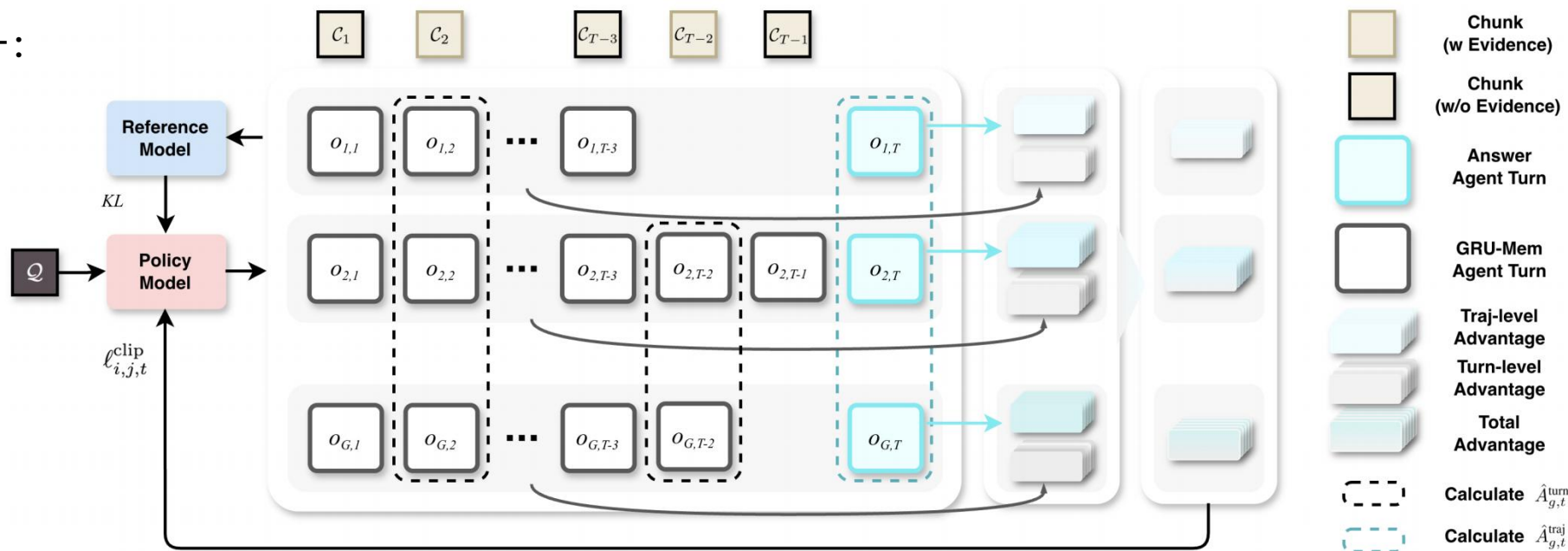


Figure 4 The advantage calculation process. The trajectory-level advantage $\hat{A}_{g,t}^{\text{traj}}$ and the turn-level advantage $\hat{A}_{g,t}^{\text{turn}}$ are calculated separately. They are combined into the total advantage with α (i.e., $\hat{A}_{g,t,i} = \alpha \hat{A}_{g,t,i}^{\text{traj}} + (1 - \alpha) \hat{A}_{g,t,i}^{\text{turn}}$).

advantage 被拆成了两层。蓝色的是 trajectory-level，反映整条 rollout 最终答得好不好、退得对不对；黑色的是 turn-level，专门监督当前这一步 update gate 的决策。

$$\hat{A}_{g,t,i}^{\text{traj}} = r_g^{\text{traj}} - \frac{1}{G} \sum_{g=1}^G r_g^{\text{traj}}, \quad \hat{A}_{g,t,i}^{\text{turn}} = r_{g,t}^{\text{update}} - \frac{1}{G_t} \sum_{g=1}^{G_t} r_{g,t}^{\text{update}}.$$

$$\hat{A}_{g,t,i} = \alpha \hat{A}_{g,t,i}^{\text{traj}} + (1 - \alpha) \hat{A}_{g,t,i}^{\text{turn}},$$

loss:

number of output tokens in this conversation. Then the overall loss can be formulated as follows:

$$\mathcal{J}(\theta) = \mathbb{E}_{(\mathcal{Q}, \mathcal{A}) \sim \mathcal{D}, \{o_{g,t}\}_{g=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | \mathcal{Q}, o_{g,t-1})} \left[\frac{1}{\sum_{g=1}^G \sum_{t=1}^{T_g} |o_{g,t}|} \sum_{g=1}^G \sum_{t=1}^{T_g} \sum_{i=1}^{|o_{g,t}|} \left(\ell_{g,t,i}^{\text{clip}} - \beta D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}}) \right) \right], \quad (3)$$

where $\ell_{g,t,i}^{\text{clip}} = \min \left(\rho_{g,t,i}(\theta) \hat{A}_{g,t,i}, \text{clip}(\rho_{g,t,i}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_{g,t,i} \right)$.

Here π_{θ} and π_{ref} denote the policy model and reference model, ε_{low} and $\varepsilon_{\text{high}}$ denote the lower and higher clipping factors as introduced in DAPO [37], and $\rho_{g,t,i}(\theta)$ refers to the importance sampling weight:

$$\rho_{g,t,i}(\theta) = \frac{\pi_{\theta}(o_{g,t,i} \mid \mathcal{Q}, o_{g,t,<i})}{\pi_{\theta_{\text{old}}}(o_{g,t,i} \mid \mathcal{Q}, o_{g,t,<i})}. \quad (4)$$

In this vanilla approach, all the conversations within one group g share the same advantage, and the advantage without the normalization of standard deviation is calculated as [20]:

$$\hat{A}_{g,t,i} = r_g^{\text{outcome}} - \text{mean}(\{r_g^{\text{outcome}}\}_{g=1}^G). \quad (5)$$

The reward for the final answer correctness is calculated as:

$$r^{\text{outcome}} = \mathbb{I}(\text{is_equiv}(\mathcal{A}, \hat{\mathcal{A}})), \quad (6)$$

inference

两种推理模式

- w/ EG：允许 exit gate 提前终止流程
 - 适合证据一旦收集充分就可回答的问题
 - 目标是提升推理效率
- w/o EG：即使预测可以退出，也仍继续处理所有chunks
 - 适合必须读完整个上下文的问题
 - 例如需要列出“所有答案”的任务

Backbone：Qwen2.5–3B–Instruct / Qwen2.5–7B–Instruct
训练数据：沿用 MemAgent 的数据设置
除 HQA 外，其余基本都是 OOD 长上下文任务

Chunk size 5000, Max prompt length 8192, Max response length 2048, Memory budget 1024.

Table 1 The performance comparison across diverse long-context tasks.

Scale	Method	Avg. Metric	Tasks									
			HQA	SQuAD	SK-1	SK-2	SK-3	MK-1	MK-2	MK-3	MQ	MV
7B	MemAgent	Perf. % ↑	76.07	79.56	99.78	95.54	97.66	97.21	75.78	95.98	88.37	81.70
		Time s ↓	463.38	162.34	378.51	420.38	419.74	412.93	349.61	403.60	419.53	407.89
	GRU-Mem (w/o EG)	Perf. % ↑	75.59	80.73	100.00	95.43	95.98	98.10	67.52	93.53	96.43	95.23
		Time s ↓	284.41	85.03	135.60	171.57	154.03	168.54	258.40	242.35	156.62	153.08
	GRU-Mem (w EG)	Perf. % ↑	76.37	80.47	100.00	96.65	95.20	98.55	84.15	95.54	84.12	-
		Time s ↓	209.33	64.32	126.00	113.89	107.64	102.46	123.53	136.02	108.62	-
3B	MemAgent	Perf. % ↑	63.87	67.58	96.76	88.73	86.72	79.46	35.05	44.42	77.26	36.27
		Time s ↓	218.60	68.81	122.31	176.48	182.72	146.77	118.16	165.48	177.22	154.78
	GRU-Mem (w/o EG)	Perf. % ↑	69.04	69.92	94.42	88.84	89.40	91.52	67.08	91.41	73.91	59.46
		Time s ↓	211.77	60.71	114.49	120.89	118.02	116.92	140.82	122.88	121.32	118.71
	GRU-Mem (w EG)	Perf. % ↑	65.33	69.66	95.31	88.28	90.85	89.84	58.15	90.85	63.84	-
		Time s ↓	162.31	45.30	104.38	84.29	82.13	74.19	59.60	68.46	80.29	-

Update Gate: 选择性更新 memory, 是 GRU-Mem 同时提升性能与效率的关键。

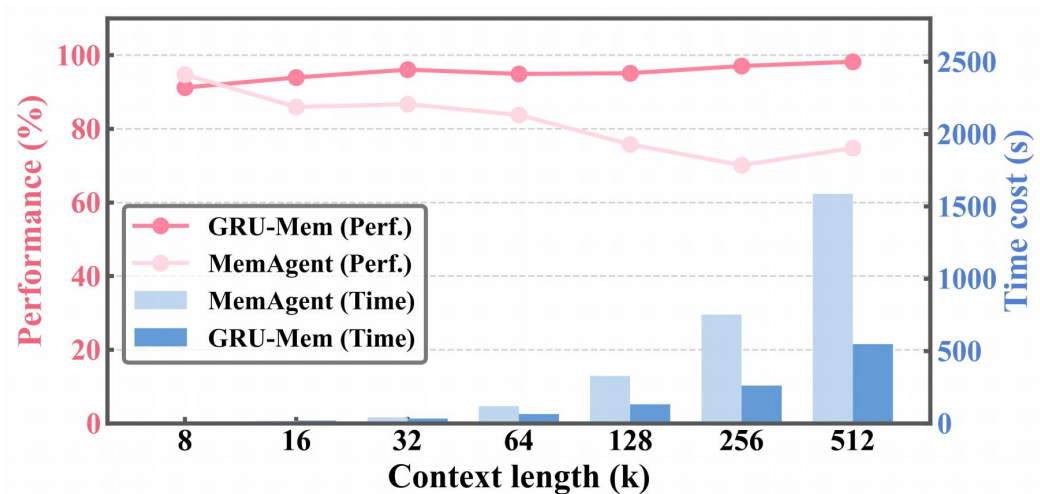


Figure 5 Performance and efficiency across diverse context lengths on the MV task.

在 MV 任务 (wo eg) 上, 随着上下文从 8K 增长到 512K, GRU-Mem 的性能始终高于 MemAgent;
同时, GRU-Mem 的推理时间显著更低, 且长度越长, 优势越明显;
说明不仅提升了效果, 也改善了长上下文推理效率;

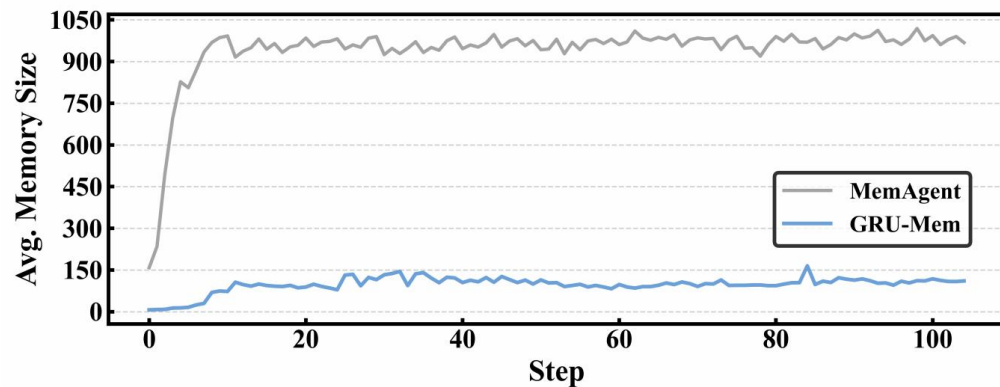


Figure 6 Memory size dynamics on MV task (512K context size).

在 512K MV 任务上, MemAgent 的 memory 很快膨胀到接近 1024 token 上限
GRU-Mem 的 memory 始终保持在较低水平, 仅在少数关键 chunk 上更新
说明 Update Gate 能有效减少无关更新, 缓解 memory explosion, 降低推理开销

Exit Gate：证据前置时的早停收益

作者构造了 evidence-unbalanced 场景，即最后一个必要证据被限制在文档前 20% 的位置出现。
性能基本保持一致：112K—896K 上，GRU-Mem 与 MemAgent 整体接近。
推理时间显著下降。

Table 2 Performance when evidence occurs at top 20% positions.

Method	Metric	Context Length			
		112K	224K	448K	896K
MemAgent	Perf. % ↑	79.69	78.91	78.12	80.47
	Time s ↓	171.65	358.60	804.23	1691.93
GRU-Mem (w EG)	Perf. % ↑	78.91	82.03	80.47	78.12
	Time s ↓	60.81	111.67	213.04	454.72

统计三类退出行为：
Early Exit / Exact Exit / Late Exit
在不同上下文长度下，Exact Exit 始终占绝大多数。
Early Exit 比例始终很低，平均仅 2.93%
说明 GRU-Mem 大多数情况下能在最后一个必要证据处正确退出，而不是盲目早停

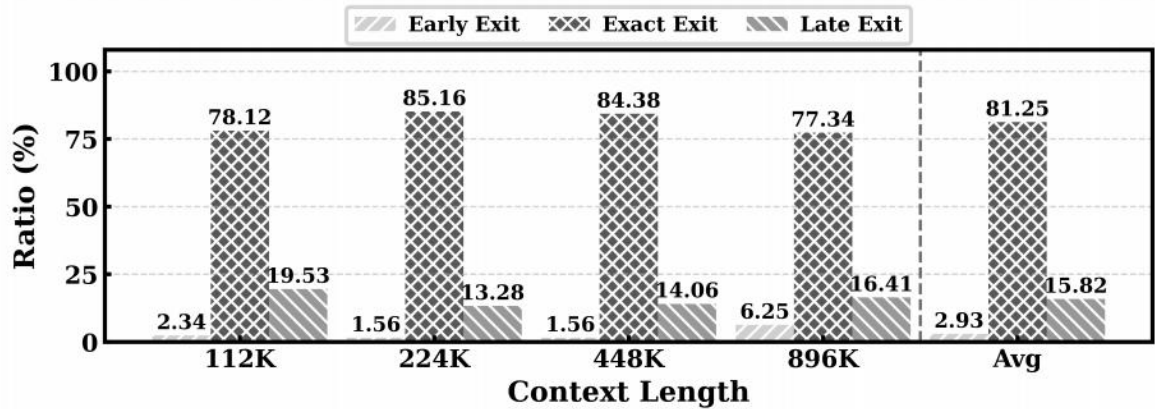
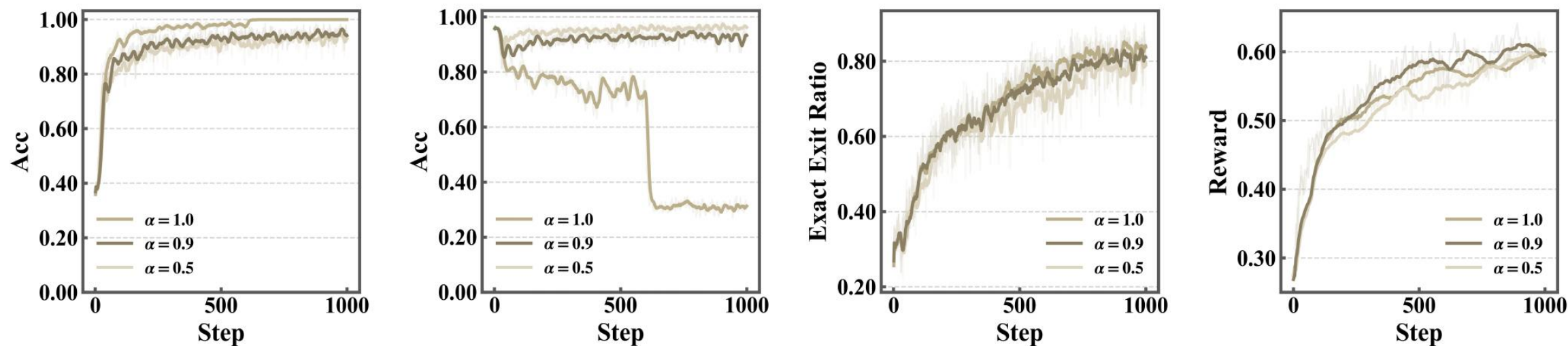


Figure 7 The ratio of early, exact, and late exit.

α 消融:



(a) Acc (evidence-present). (b) Acc (evidence-free). (c) Ratio of exactly exiting. (d) Validation Reward.

Figure 8 Training dynamics of update and exit gates.

较大的 α 更有利于 evidence-present chunk 上的更新
但同时会增加 evidence-free chunk 上的不必要更新风险

当 $\alpha = 1$ 时, 没有 update reward, 模型更容易 indiscriminate update
 $\alpha = 0.9$ 在更新准确率、exact exit ratio 和验证集 reward 上更均衡、更稳定
因此作者最终采用 $\alpha = 0.9$ 作为默认设置

总结与思考：

优点：

视角新，memory 的核心不一定是保存所有信息，而是围绕当前决策，持续提取必要信息。

框架清晰：固定长度 textual memory，使方法天然具有长文本外推和线性复杂度优势。

GRU-Mem 的 Update Gate 很合理：它把“是否值得写入 memory”显式化，确实能减少无关 chunk 带来的噪声累积。

不足：

适用范围偏窄：更像“长文本 + 单一 query”的证据提取框架，而不是可复用的通用 memory。因为一旦问题变化，memory 往往不能直接复用，需要重新过所有 chunk。

Exit Gate 的泛化性存疑：作者自己也承认，有些任务必须读完整个上下文，比如 multi-value 这类需要找全答案的任务，因此专门保留了 w/o EG 的推理模式。

如果放到真实对话或 agent 场景里，后续问题、偏好和信息需求可能变化，因此“当前证据已经足够，可以提前停”这个假设未必成立。这个点在静态单问答任务里成立，但在开放式交互里不一定能泛化。